



UNIVERSIDAD POLITÉCNICA SALESIANA

CARRERA DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

TICKET RESILIENCE LAB

Práctica: mecanismos de tolerancia a fallas en Kubernetes

Integrantes: Alexander Chuquipoma y Jean Pierre Valarezo

Materia: Sistemas Distribuidos

Docente: Ing. Cristian Timbi

Periodo académico: 2026-2027

Fecha: 14/07/2026

Cuenca - Ecuador

Resumen

Este informe documenta una práctica de tolerancia a fallas aplicada a un sistema simplificado de reservas de entradas desplegado en Kubernetes. La solución conserva seis componentes solicitados y ejecuta cuatro fallos reales sobre un cluster de dos nodos: inventario caído, pasarela lenta, notificaciones caídas y diluvio de peticiones. Los dos fallos restantes se analizan teóricamente con propuestas de solución de nivel producción.

Palabras clave: Kubernetes, tolerancia a fallas, microservicios, resiliencia, circuit breaker, retry, fallback, rate limiting, consistencia.

1. Objetivo, alcance y criterios

El objetivo fue construir una práctica reproducible en la que los fallos se provoquen sobre infraestructura Kubernetes real. El alcance cubre despliegue, scripts de inyeccion, evidencias, demo y analisis de los escenarios restantes.

Criterio	Cumplimiento
Cluster de al menos dos nodos	Perfil minikube ticket-lab con nodos ticket-lab y ticket-lab-m02.
Componente crítico replicado	inventory con 2 réplicas distribuidas por topologySpreadConstraints.
Seis componentes	gateway, reservations, inventory, payments, notifications y postgres.
Cuatro fallos implementados	Inventory down, payments lento, notifications down y traffic spike.
Dos fallos analizados	Base de datos intermitente y condición de carrera.
Evidencia y demo	Capturas en docs/evidence y scripts numerados en scripts/.

2. Arquitectura y despliegue

Componente	Rol	Responsabilidad	Deployment
API Gateway	Entrada del sistema	Recibe solicitudes de clientes y aplica rate limiting.	gateway
Reservas Core	Orquestacion	Coordina inventario, pago y notificación.	reservations
Inventario	Consistencia crítica	Valida y reserva asientos con PostgreSQL.	inventory
Pagos	Stub externo	Simula latencia y fallos realistas de una pasarela.	payments
Notificaciones	Stub no crítico	Simula envio de confirmaciones por correo.	notifications
Base de datos	Persistencia	Almacena estado de asientos y soporta bloqueo transaccional.	postgres

Arquitectura y distribucion multi-nodo

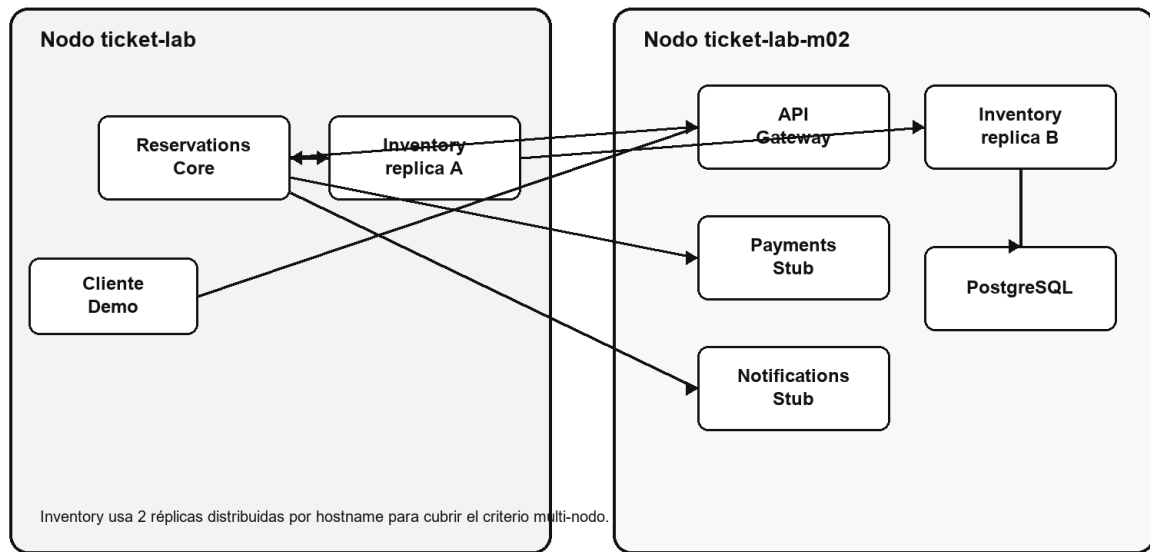


Figura 1. Arquitectura y distribucion multi-nodo del laboratorio.

El despliegue usa Services internos de Kubernetes, endpoints /health y readiness/liveness probes. Inventory se replica en dos nodos para evitar concentrar todo el componente crítico en una sola maquina.

3. Catálogo de fallos y matriz técnica

Fallo	Tipo	Inyeccion	Defensa	Estado
Inventario Fantasma	Disponibilidad	Escalar deployment/inventory a 0 réplicas	Retry con backoff y fallo controlado	Implementado
Pasarela Lenta	Latencia	PAYMENT_DELAY_MS=20000	Timeout + circuit breaker	Implementado
Diluvio de Peticiones	Sobrecarga	25 solicitudes consecutivas contra POST /reserve	Rate limiting en gateway	Implementado
Base de Datos Intermitente	Conectividad	Flapping de conexion o reinicio controlado de PostgreSQL	Failover, backoff, idempotencia y outbox	Análisis teórico
Correo Perdido	Fallo no crítico	Escalar deployment/notifications a 0 réplicas	Fallback con notification.status=pending	Implementado
Condición de Carrera	Consistencia	Solicitudes concurrentes sobre el mismo asiento	SELECT FOR UPDATE, holds e idempotencia	Análisis teórico

4. Mecanismos implementados

Inventario Fantasma

Riesgo tecnico: Sin inventario, el sistema podria confirmar una reserva sin comprobar disponibilidad real.

Comando: `.\scripts\05-chaos-inventory-down.ps1`

Defensa: El servicio reservations aplica retry con backoff al invocar inventory. Si la dependencia no responde, se devuelve un error controlado y no se continua con pago ni notificación.

Justificacion: Retry con backoff es adecuado porque una caída breve de pods o endpoints puede ser transitoria. No se usa fallback positivo porque inventario es una dependencia crítica: inventar disponibilidad seria técnicamente incorrecto.

Resultado: La evidencia muestra HTTP 503. El sistema falla rapido y no confirma una reserva fantasma.

Pasarela Lenta

Riesgo tecnico: Una pasarela de pagos lenta puede dejar hilos/conexiones esperando y propagar latencia al usuario.

Comando: `.\scripts\07-chaos-payments-slow.ps1`

Defensa: reservations usa timeout al llamar payments y mantiene un circuit breaker en memoria. Tras tres fallos, el circuito pasa a open y rechaza nuevas llamadas sin esperar los 20 segundos de latencia.

Justificacion: El timeout limita la espera por solicitud y el circuit breaker evita seguir llamando una dependencia degradada. Unicamente reintentar seria peligroso porque amplificaria la carga sobre payments.

Resultado: Los intentos devuelven HTTP 503 y /resilience reporta payment_circuit.state=open.

Correo Perdido

Riesgo tecnico: Una falla de notificaciones podria cancelar ventas ya pagadas si se trata como dependencia crítica.

Comando: `.\scripts\09-chaos-notifications-down.ps1`

Defensa: notifications se considera dependencia no crítica. Si no responde, reservations conserva la reserva confirmada y marca la notificación como pending para reintento operativo posterior.

Justificacion: Fallback es el patrón correcto porque el correo no define la validez de la entrada. Cancelar la reserva por no enviar email degradaria la disponibilidad sin proteger consistencia.

Resultado: La evidencia muestra HTTP 200, status=confirmed y notification.status=pending.

Diluvio de Peticiones

Riesgo tecnico: Un pico de tráfico puede saturar el gateway y arrastrar servicios internos.

Comando: `.\scripts\11-chaos-traffic-spike.ps1`

Defensa: gateway mantiene un contador temporal por cliente y corta el exceso de solicitudes con HTTP 429. Los conflictos funcionales se reportan como HTTP 409, separados de la sobrecarga.

Justificacion: Rate limiting debe ubicarse en el borde para rechazar pronto el exceso de tráfico. Un circuit breaker interno no evitaria que las solicitudes entren al sistema.

Resultado: La evidencia muestra respuestas HTTP 429 despues de superar el limite configurado.

5. Guion de demo en vivo

Tiempo	Responsable	Accion	Evidencia esperada
0:00-2:00	Alexander	Mostrar nodos y pods.	Dos nodos Ready e inventory replicado.
2:00-4:00	Jean Pierre	Ejecutar flujo base.	HTTP 200 y reserva confirmed.
4:00-6:30	Alexander	Inventory down y recuperacion.	HTTP 503 controlado.

Tiempo	Responsable	Accion	Evidencia esperada
6:30-9:00	Jean Pierre	Payments lento.	Circuit breaker open.
9:00-11:30	Alexander	Notifications down.	Reserva confirmed con pending.
11:30-13:30	Jean Pierre	Traffic spike.	HTTP 429 en gateway.
13:30-15:00	Ambos	Conclusiones y límites.	Relación con fallos teoricos.

6. Análisis teórico de fallos restantes

6.1 Base de Datos Intermitente

Este fallo ocurre cuando inventory pierde conectividad intermitente con PostgreSQL durante escrituras. Desde CAP, una particion obliga a decidir entre disponibilidad y consistencia; en reservas, confirmar sin escritura durable puede crear ventas fantasma.

Solucion propuesta: base de datos con failover, readiness probes, pool de conexiones, retries con backoff y jitter, idempotency keys y patrón outbox.

Pseudocodigo: begin transaction; select seat for update; si esta disponible, crear reserva con idempotency key y evento outbox; commit; ante error transitorio, rollback y retry; si excede el limite, error controlado.

6.2 Condición de Carrera

La condición de carrera ocurre cuando dos usuarios intentan comprar el mismo asiento al mismo tiempo. Sin aislamiento transaccional, ambos procesos pueden observar disponibilidad y confirmar ventas duplicadas.

Solucion propuesta: transacciones ACID, SELECT FOR UPDATE, restriccion unica por asiento activo, holds temporales con expiracion e idempotencia por solicitud.

Pseudocodigo: begin transaction; select seat for update; si status != available, rollback y conflict; crear hold con expires_at; update seat=held; commit. Para confirmar, bloquear hold, validar expiracion y marcar seat=reserved.

7. Reproducibilidad

Paso	Comando
Preparar cluster	.\scripts\00-minikube-ensure-two-nodes.ps1
Construir imagenes	.\scripts\01-build-load-images.ps1
Desplegar	.\scripts\02-deploy-k8s.ps1
Port-forward	.\scripts\03-port-forward.ps1
Evidencia base	.\scripts\04-baseline-evidence.ps1
Reiniciar laboratorio	.\scripts\12-reset-lab.ps1

8. Declaración de uso de inteligencia artificial

Durante el desarrollo se utilizó ChatGPT 5.5 como apoyo para organizar la práctica, revisar la correspondencia con la rúbrica, proponer la estructura del informe, depurar comandos y mejorar la redacción técnica. La herramienta no sustituyó la ejecución práctica: los comandos fueron corridos sobre el entorno local, las capturas fueron generadas por los integrantes y los resultados se verificaron contra el cluster Kubernetes desplegado.

Se revisaron manualmente los resultados antes de incorporarlos al informe. Las decisiones técnicas principales fueron implementar cuatro fallos en vivo, dejar dos como analisis teorico, usar inventory como componente crítico replicado y separar errores de negocio HTTP 409 de protección por sobrecarga HTTP 429.

9. Conclusiones

La práctica demuestra que cada patrón debe ubicarse en el punto correcto: retry con backoff para dependencias transitorias, timeout y circuit breaker frente a latencia sostenida, fallback para dependencias no críticas y rate limiting en el borde del sistema.

Referencias bibliográficas

Brewer, E. A. (2012). CAP twelve years later: How the rules have changed. *Computer*, 45(2), 23-29.
<https://doi.org/10.1109/MC.2012.37>

Fowler, M. (2014). Circuit Breaker. <https://martinfowler.com/bliki/CircuitBreaker.html>

Kubernetes. (2026). Deployments. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>

Kubernetes. (2026). Services, Load Balancing, and Networking. <https://kubernetes.io/docs/concepts/services-networking/>

Kubernetes. (2026). Configure Liveness, Readiness and Startup Probes.
<https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>

Kubernetes. (2026). Pod Topology Spread Constraints.
<https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/>

Kubernetes. (2026). Horizontal Pod Autoscaling. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>

Nygard, M. T. (2018). *Release It! Design and Deploy Production-Ready Software* (2nd ed.). Pragmatic Bookshelf.

PostgreSQL Global Development Group. (2026). Explicit Locking. <https://www.postgresql.org/docs/current/explicit-locking.html>

PostgreSQL Global Development Group. (2026). SELECT. <https://www.postgresql.org/docs/current/sql-select.html>

Anexo A. Evidencias finales

```
PS C:\Users\Pc\Documents\ticket-resilience-lab> kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE     VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
KERNEL-VERSION                     CONTAINER-RUNTIME
ticket-lab                          Ready    control-plane  3h48m   v1.35.1   192.168.58.2   <none>        Debian GNU/Linux 12 (bookworm)
6.6.87.2-microsoft-standard-WSL2   docker://29.2.1
ticket-lab-m02                      Ready    <none>     3h47m   v1.35.1   192.168.58.3   <none>        Debian GNU/Linux 12 (bookworm)
6.6.87.2-microsoft-standard-WSL2   docker://29.2.1
PS C:\Users\Pc\Documents\ticket-resilience-lab> kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE     VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE
KERNEL-VERSION                     CONTAINER-RUNTIME
ticket-lab                          Ready    control-plane  4h2m    v1.35.1   192.168.58.2   <none>        Debian GNU/Linux 12 (bookworm)
6.6.87.2-microsoft-standard-WSL2   docker://29.2.1
ticket-lab-m02                      Ready    <none>     4h1m    v1.35.1   192.168.58.3   <none>        Debian GNU/Linux 12 (bookworm)
6.6.87.2-microsoft-standard-WSL2   docker://29.2.1
PS C:\Users\Pc\Documents\ticket-resilience-lab> |
```

Cluster Kubernetes con dos nodos en estado Ready.

```
PS C:\Users\Pc\Documents\ticket-resilience-lab> kubectl -n ticket-lab get pods -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP             NODE             NOMINATED NODE    REA
DINESS GATES
gateway-6c849dccc6-48nm2            1/1     Running   0           4h31m   10.244.1.2     ticket-lab-m02   <none>            <no
ne>
inventory-7d6f5868b7-hh7ss          1/1     Running   0           4m15s   10.244.0.7     ticket-lab        <none>            <no
ne>
inventory-7d6f5868b7-v8j89          1/1     Running   0           4m27s   10.244.1.17    ticket-lab-m02   <none>            <no
ne>
notifications-7699c97d7-d2bt2       1/1     Running   0           18m     10.244.1.16    ticket-lab-m02   <none>            <no
ne>
payments-559ff975d5-zvndx           1/1     Running   0           20m     10.244.1.14    ticket-lab-m02   <none>            <no
ne>
postgres-754697f674-68w96          1/1     Running   0           4h31m   10.244.1.7     ticket-lab-m02   <none>            <no
ne>
reservations-db959d9c4-shq5r        1/1     Running   0           20m     10.244.1.15    ticket-lab-m02   <none>            <no
ne>
```

Pods desplegados: inventory aparece replicado entre ticket-lab y ticket-lab-m02.

```
PS C:\Users\Pc\Documents\ticket-resilience-lab> .\scripts\03-port-forward.ps1

Port-forwards iniciados. PIDs guardados en C:\Users\Pc\Documents\ticket-resilience-lab\.run\port-forward-pids.txt
gateway,11024
reservations,34324
inventory,20952

URLs locales:
- Gateway:      http://localhost:8080
- Reservations: http://localhost:8081
- Inventory:    http://localhost:8082
```

Port-forward hacia gateway, reservations e inventory.

```
PS C:\Users\Pc\Documents\ticket-resilience-lab> .\scripts\04-baseline-evidence.ps1

=== Gateway health ===
HTTP 200
{
  "status": "ok",
  "service": "gateway"
}

=== Inventory health ===
HTTP 200
{
  "status": "ok",
  "service": "inventory"
}

=== Inventario antes de reservar ===
HTTP 200
{
  "event_id": "concert-1",
  "seats": [
    {
      "seat_id": "A1",
      "status": "reserved",
      "reserved_by": "demo-user"
    },
    {
      "seat_id": "A10",
      "status": "reserved",
      "reserved_by": "flood-demo-1"
    }
  ]
}
```

Health checks e inventario inicial.


```

        {
            "seat_id": "A8",
            "status": "available",
            "reserved_by": null
        },
        {
            "seat_id": "A9",
            "status": "available",
            "reserved_by": null
        }
    ]
}

=== Reserva exitosa via gateway ===
HTTP 200
{
    "reservation_id": "2386d5a0-8546-450b-8d25-83ef06a1a961",
    "status": "confirmed",
    "inventory": {
        "event_id": "concert-1",
        "seat_id": "A2",
        "status": "reserved",
        "reserved_by": "demo-user"
    },
    "payment": {
        "status": "charged",
        "transaction_id": "d46b1582-aa97-424f-b541-caa87db87625",
        "reservation_id": "2386d5a0-8546-450b-8d25-83ef06a1a961",
        "amount": 50.0
    }
},

```

Reserva base confirmada.

```

    },
    "notification": {
        "status": "sent"
    }
}

=== Inventario despues de reservar ===
HTTP 200
{
    "event_id": "concert-1",
    "seats": [
        {
            "seat_id": "A1",
            "status": "reserved",
            "reserved_by": "demo-user"
        },
        {
            "seat_id": "A10",
            "status": "reserved",
            "reserved_by": "flood-demo-1"
        },
        {
            "seat_id": "A2",
            "status": "reserved",
            "reserved_by": "demo-user"
        },
        {
            "seat_id": "A3",
            "status": "available",
            "reserved_by": null
        }
    ]
}

```

Inventario posterior con asiento reservado.

```

PS C:\Users\Pc\Documents\ticket-resilience-lab> .\scripts\05-chaos-inventory-down.ps1
Injectando falla: Inventory down.
deployment.apps/inventory scaled
NAME                                READY    STATUS    RESTARTS   AGE    IP             NODE             NOMINATED NODE    REA
DINESS GATES
gateway-6c849dccc6-48nm2            1/1     Running   0           4h6m   10.244.1.2     ticket-lab-m02    <none>             <no
ne>
notifications-7699c97d7-z2x6t       1/1     Running   0           3h59m   10.244.1.11    ticket-lab-m02    <none>             <no
ne>
payments-559ff975d5-59lqz          1/1     Running   0           4h     10.244.1.9     ticket-lab-m02    <none>             <no
ne>
postgres-754697f674-68w96          1/1     Running   0           4h6m   10.244.1.7     ticket-lab-m02    <none>             <no
ne>
reservations-6c89b446cc-ss5dn       1/1     Running   0           3h58m   10.244.0.5     ticket-lab        <none>             <no
ne>

=== Reserva con Inventory caido ===
HTTP 503

```

Inventory down con HTTP 503.

```

PS C:\Users\Pc\Documents\ticket-resilience-lab> .\scripts\07-chaos-payments-slow.ps1
Inyectando falla: pasarela de pagos lenta con 20 segundos de latencia.
deployment.apps/payments env updated
Waiting for deployment "payments" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "payments" rollout to finish: 1 old replicas are pending termination...
deployment "payments" successfully rolled out

=== Intento 1 con pagos lentos ===
HTTP 503

=== Intento 2 con pagos lentos ===
HTTP 503

=== Intento 3 con pagos lentos ===
HTTP 503

=== Intento 4 con pagos lentos ===
HTTP 503

=== Estado del circuit breaker de pagos ===
HTTP 200
{
  "payment_circuit": {
    "state": "open",
    "failure_count": 3
  }
}

```

Payments lento con circuit breaker abierto.

```

PS C:\Users\Pc\Documents\ticket-resilience-lab> .\scripts\09-chaos-notifications-down.ps1
Inyectando falla: Notifications down.
deployment.apps/notifications scaled
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
gateway-8655c8f889-lcch5 1/1 Running 0 40s 10.244.1.29 ticket-lab-m02 <none> <none>
inventory-5544cd6f6-cjq8f 1/1 Running 0 27s 10.244.1.30 ticket-lab-m02 <none> <none>
inventory-5544cd6f6-sllcf 1/1 Running 0 40s 10.244.0.16 ticket-lab <none> <none>
payments-5d6ff89484-4zvw7 1/1 Running 0 40s 10.244.1.28 ticket-lab-m02 <none> <none>
postgres-7d4fd86d89-xfcq6 1/1 Running 0 40s 10.244.1.27 ticket-lab-m02 <none> <none>
reservations-6874999577-k7tdn 1/1 Running 0 40s 10.244.0.17 ticket-lab <none> <none>

=== Reserva con Notifications caído ===
HTTP 200
{
  "reservation_id": "a0a213c5-8434-440c-a48c-bfadfa38fddd",
  "status": "confirmed",
  "inventory": {
    "event_id": "concert-1",
    "seat_id": "A7",
    "status": "reserved",
    "reserved_by": "notification-down-demo"
  },
  "payment": {
    "status": "charged",
    "transaction_id": "3beff8c8-a2aa-4c0d-97f6-3b68be2fcf04",
    "reservation_id": "a0a213c5-8434-440c-a48c-bfadfa38fddd",
    "amount": 50.0
  },
  "notification": {
    "status": "pending",
    "reason": "notification service unavailable"
  }
}

Resultado esperado: reserva confirmada y notification.status=pending.

```

Notifications caído con fallback pending.

```
PS C:\Users\Pc\Documents\ticket-resilience-lab> .\scripts\11-chaos-traffic-spike.ps1
Inyectando falla: diluvio de peticiones contra el gateway.
Se esperan respuestas 429 cuando el rate limiter corta el exceso de trafico.
Intento 1: HTTP 200
Intento 2: HTTP 409
Intento 3: HTTP 409
Intento 4: HTTP 409
Intento 5: HTTP 409
Intento 6: HTTP 409
Intento 7: HTTP 409
Intento 8: HTTP 409
Intento 9: HTTP 409
Intento 10: HTTP 409
Intento 11: HTTP 409
Intento 12: HTTP 409
Intento 13: HTTP 429
Intento 14: HTTP 429
Intento 15: HTTP 429
Intento 16: HTTP 429
Intento 17: HTTP 429
Intento 18: HTTP 429
Intento 19: HTTP 429
Intento 20: HTTP 429
Intento 21: HTTP 429
Intento 22: HTTP 429
Intento 23: HTTP 429
Intento 24: HTTP 429
Intento 25: HTTP 429
```

Resumen por codigo HTTP:

Name	Count
200	1
409	11
429	13

Traffic spike con HTTP 429.